# Waffle 5.1 fast (high) — PO Yönetim Sistemi

**CSE 321 Data Systems — Data & System Architecture Raporu** · 12 Kasım 2026

**Ekip:** EmreSndkc · brkznqr · eminozkanca · Berkegn

**Repo:** https://github.com/EmreSndkc/waffle_v5.1

## 1 Introduction

Waffle 5.1, Acıbadem Üniversitesi CSE bölümü için her öğrencinin 11 Program Outcome (PO) puanını hesaplayan, Python + Django ile yazılmış bir web sistemidir. Hesap zinciri üç basamaklıdır: değerlendirme notu → (yüzdelerle) Learning Outcome (LO) → (1–5 ağırlıkla) PO. Her basamakta aynı formül kullanılır:

```
puan = Σ(ağırlık × puan) ÷ Σ ağırlık
```

### Temel özellikler (bugünkü durum)

| Özellik | Durum |
|---|---|
| Veri modeli: kullanıcı/rol, ders, kayıt, LO, PO, değerlendirme, not, eşlemeler | Çalışıyor |
| PO hesaplama servisi ve ön hesaplanmış PO profili (`StudentPOSnapshot`) | Çalışıyor |
| Not, eşleme ya da ağırlık değişince otomatik yeniden hesaplama | Çalışıyor |
| Verisi olmayan PO için 0 yerine NULL ("veri yok") | Çalışıyor |
| Django admin paneli: tüm modeller, liste/filtre/arama | Çalışıyor |
| Demo verisi: `python manage.py seed_demo` (11 PO, 3 ders, her rol için kullanıcı) | Çalışıyor |
| Rol bazlı ekranlar, öğrenci paneli, toplu not yükleme, raporlar | Sonraki sprintler |

### Kabul testi (hocanın örneği)

Midterm 90, Project 80 alan öğrenci için sistem LO1 = 86.0, LO2 = 82.9, PO1 = 85.0, PO2 = 82.9 üretir. Bu örnek hem birim testidir hem de demo verisindeki `elif` kullanıcısıdır.

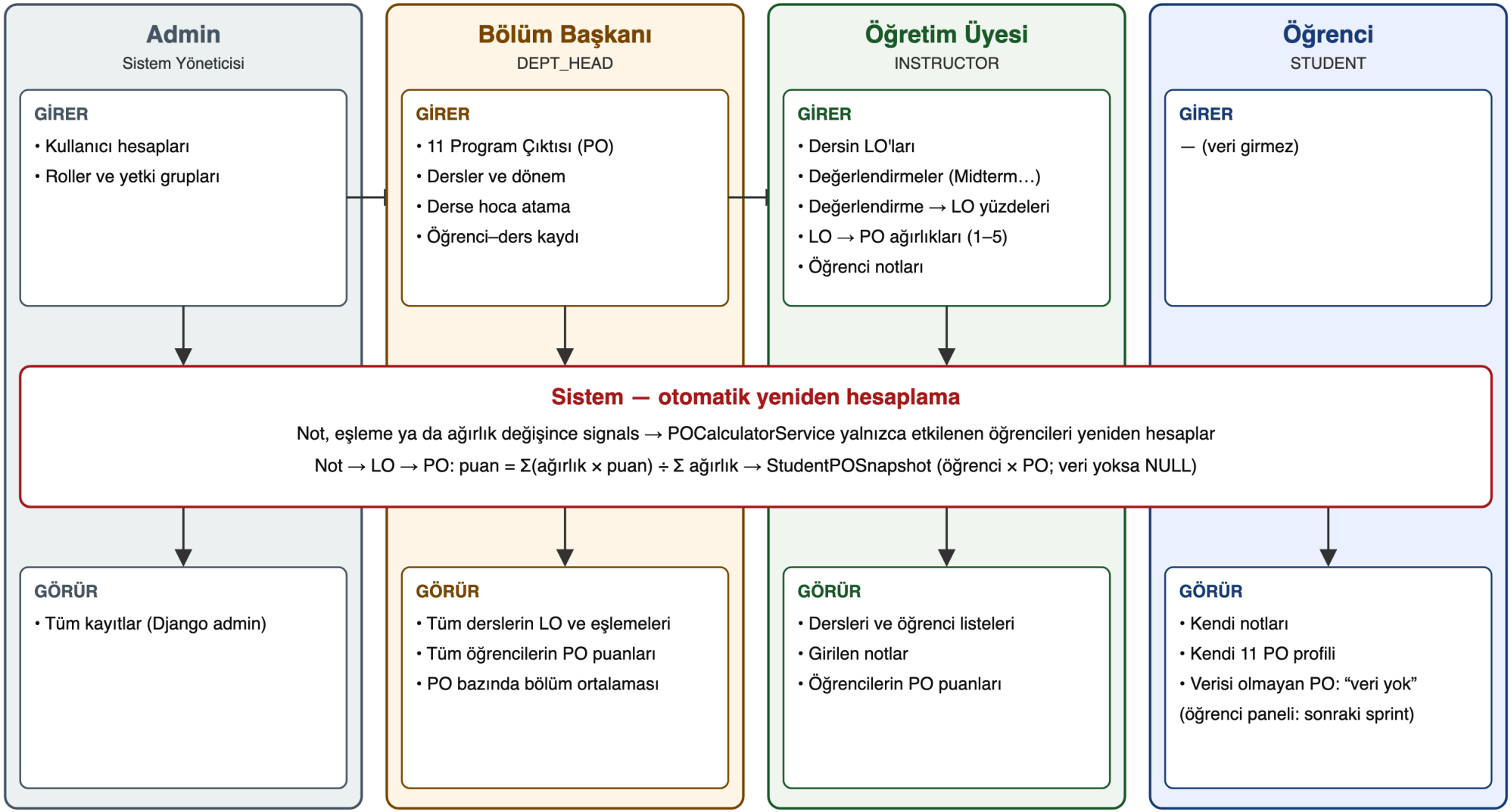
### Roller

| Rol | Kod | Girer | Görür |
|---|---|---|---|
| Sistem Yöneticisi | `ADMIN` | Kullanıcılar, roller | Her şey |
| Bölüm Başkanı | `DEPT_HEAD` | PO'lar, dersler, hoca ataması, öğrenci kaydı | Tüm dersler ve tüm öğrencilerin PO puanları |
| Öğretim Üyesi | `INSTRUCTOR` | LO'lar, değerlendirmeler, yüzdeler, LO→PO ağırlıkları, notlar | Dersler, notlar, PO puanları |
| Öğrenci | `STUDENT` | — | Kendi notları ve PO profili |

Kural: kurumsal yapıyı (PO, ders) bölüm, ders içeriğini (LO, ağırlık, not) hoca girer; öğrenci yalnızca kendi verisini görür. Bugün Bölüm Başkanı ve Öğretim Üyesi yetkileri Django admin'de

iki yetki grubu olarak tanımlıdır. Yetki model düzeyindedir; "hoca yalnızca kendi dersini görür" kontrolü ve öğrenci paneli henüz yoktur.

## Waffle 5.1 — Roller: Kim Neyi Girer, Kim Neyi Görür




**Şekil 1 — Roller: kim neyi girer, kim neyi görür**

# 2 System Design

## 2.1 Sistem mimarisi

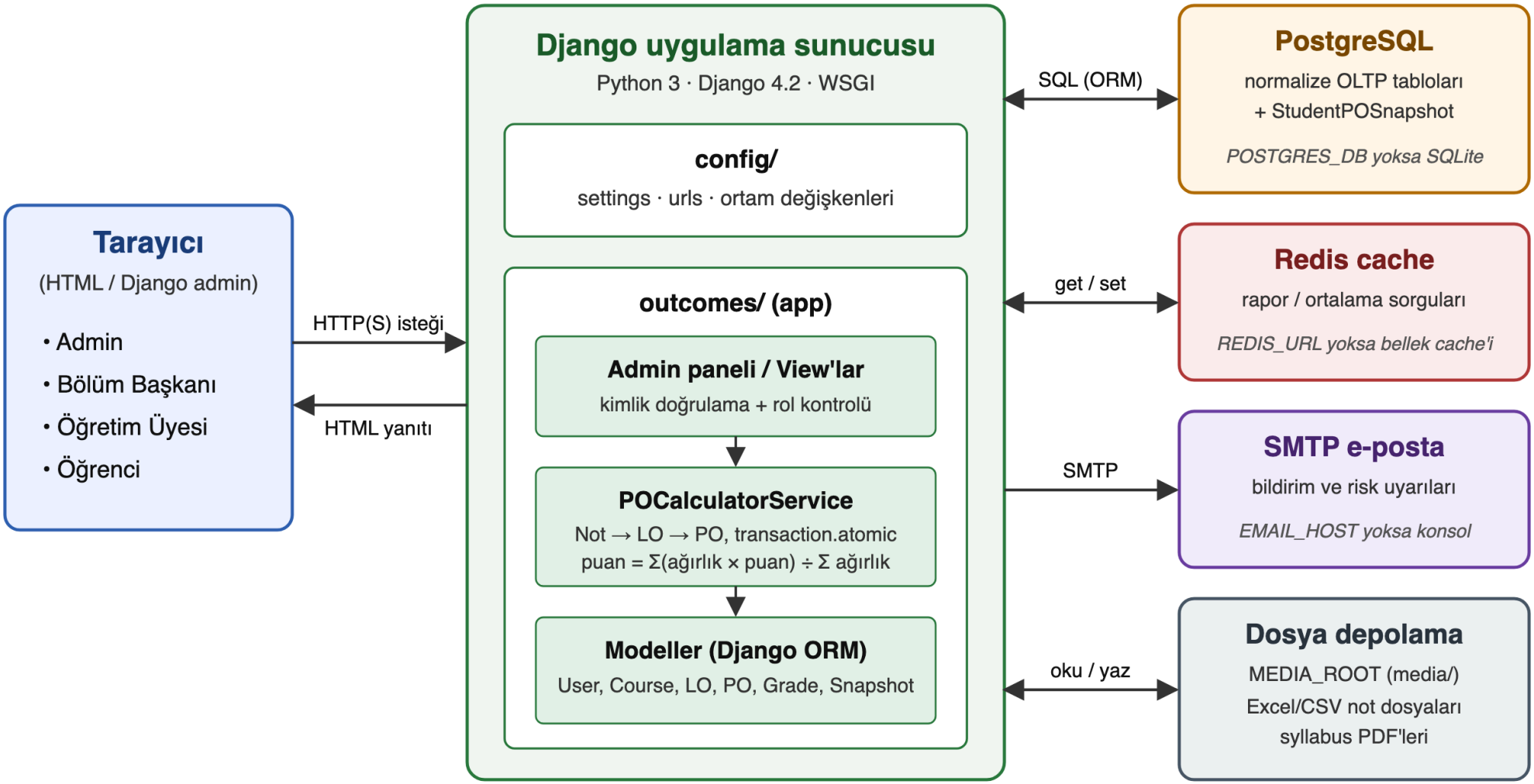
Sistem tek bir Django uygulama sunucusudur. Tarayıcıdan gelen istek `config/` altındaki URL ve ayarlardan geçer, `outcomes/` uygulamasında yetki kontrolünden sonra modellere ve hesaplama servisine ulaşır. Dış bileşenlerin tamamı `config/settings.py` içinde ortam değişkenleriyle seçilir; değişken tanımlı değilse yerel bir karşılığa düşer, böylece proje ek kurulum olmadan çalışır.

| Bileşen | Görevi | Ortam değişkeni | Tanımlı değilse | Bugünkü durum |
|---|---|---|---|---|
| PostgreSQL | Kalıcı veri | `POSTGRES_DB` … | SQLite | Kullanılıyor; testler PostgreSQL 18'de geçiyor |
| Redis | Rapor sorgularının cache'i | `REDIS_URL` | Süreç içi bellek | Yapılandırıldı; henüz cache'lenen sorgu yok |
| SMTP | E-posta bildirimi | `EMAIL_HOST` … | Konsola yazar | Yapılandırıldı; henüz e-posta gönderen özellik yok |
| Dosya depolama | Excel/CSV not dosyaları, syllabus PDF | `MEDIA_ROOT` | `./media` | Yapılandırıldı; yükleme ekranı henüz yok |

Veritabanı seçimi tek bir koşuldur:

```
if os.environ.get("POSTGRES_DB"):
    DATABASES = {"default": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": os.environ["POSTGRES_DB"],
        "ATOMIC_REQUESTS": True,   # her HTTP isteği tek transaction
        ...
    }}
else:
    DATABASES = {"default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "db.sqlite3",
        "ATOMIC_REQUESTS": True,
    }}
```

# Waffle 5.1 — Sistem Mimarisi




**Şekil 2 — Sistem mimarisi**

### 2.2 Class diyagramı

Kod üç parçadır: modeller (`outcomes/models.py`), hesaplama servisi (`outcomes/services.py`) ve tetikleyiciler (`outcomes/signals.py`).

| Sınıf | Sorumluluğu |
|---|---|
| `User`, `Course`, `Enrollment`, `LearningOutcome`, `ProgramOutcome` | Kurumsal yapı: kim, hangi ders, hangi çıktı |
| `Assessment`, `StudentGrade` | Değerlendirmeler ve ham notlar |
| `Assessment_LO_Mapping`, `LO_PO_Mapping` | Ağırlıklar: değerlendirme → LO (katsayı), LO → PO (1–5) |
| `StudentPOSnapshot` | Ön hesaplanmış PO profili; yalnızca servis yazar |
| `POCalculatorService` | Not → LO → PO hesabı, snapshot yazımı, etkilenen öğrencilerin bulunması |
| `signals` | Not, eşleme ya da ağırlık değişince servisi çağırır |

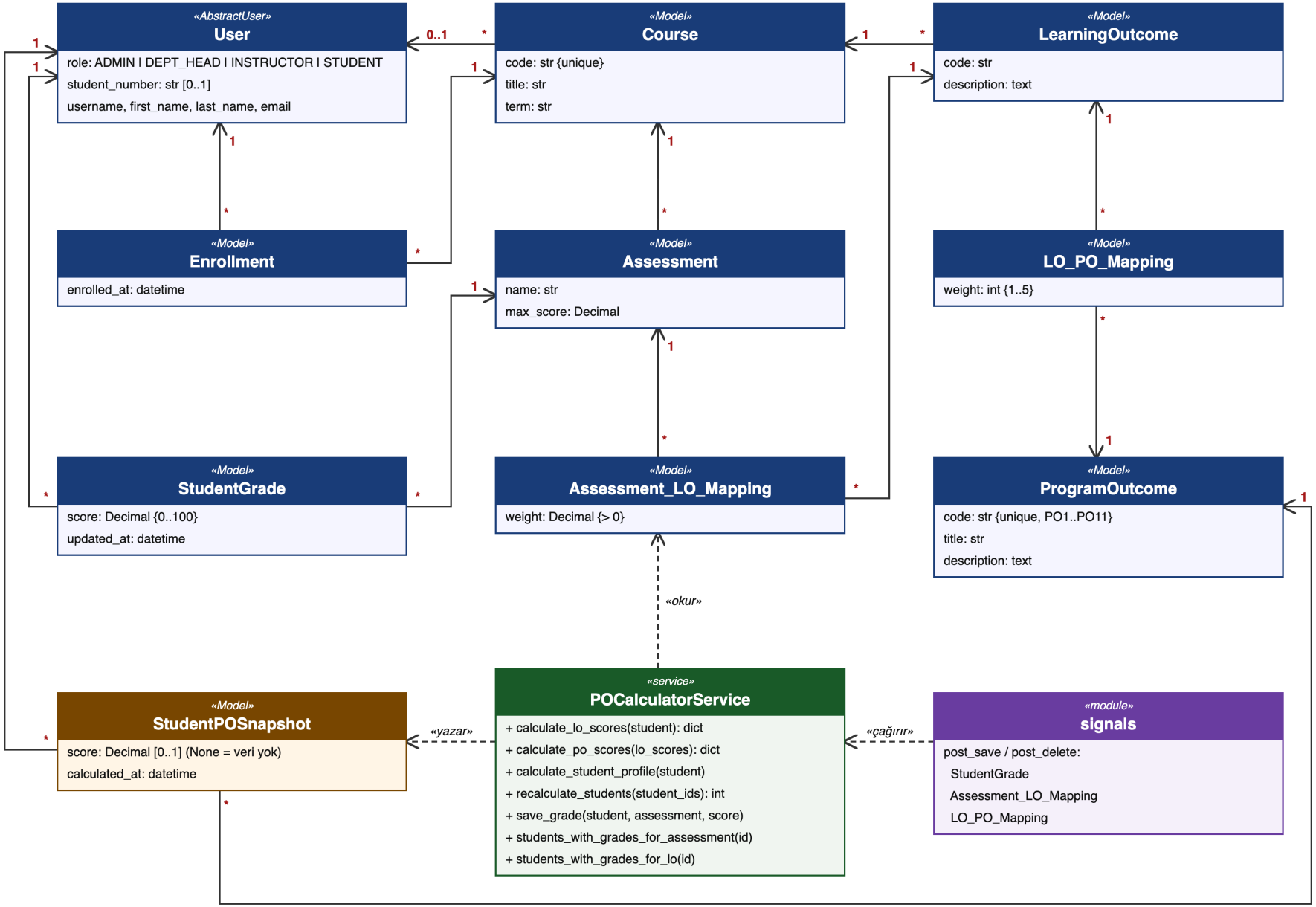
Formül tek bir fonksiyondadır ve iki adımda da aynı fonksiyon kullanılır:

```
def weighted_average(pairs):
    weighted_sum, total_weight = Decimal('0'), Decimal('0')
    for weight, score in pairs:
        weighted_sum += Decimal(weight) * Decimal(score)
        total_weight += Decimal(weight)
    if total_weight == 0:
        return None            # veri yok: 0 değil
    return weighted_sum / total_weight
```

### Hesaplama örneği

| Adım | Hesap | Sonuç |
|---|---|---|
| LO1 | (0.60 × 90 + 0.40 × 80) ÷ (0.60 + 0.40) | 86.0 |
| LO2 | (0.40 × 90 + 1.00 × 80) ÷ (0.40 + 1.00) | 82.9 |
| PO1 | (2 × 86.0 + 1 × 82.86) ÷ (2 + 1) | 85.0 |
| PO2 | (2 × 82.86) ÷ 2 | 82.9 |
| PO3 | bağlı LO yok | NULL |

Hocanın örneğindeki LO→PO ağırlıkları 1.0 : 0.5 : 1.0'dır. Modelde ağırlık 1–5 tam sayıdır, bu yüzden aynı oran 2 : 1 : 2 olarak girilir. Ağırlıklı ortalamada yalnızca oran önemli olduğundan sonuç aynıdır.




Düz ok: ilişki (ForeignKey), * → 1. Kesikli ok: bağımlılık. signals, kayıt değişince servisi çağırır; servis kaynak tabloları okur, snapshot'ı yazar.

**Şekil 3 — Class diyagramı**

# 3 Data Model

Şema on tablodan oluşur. Dokuz kaynak tablo 3NF'tedir: her not ve her ağırlık tek bir yerde durur. Onuncu tablo `StudentPOSnapshot` türetilmiş veridir (öğrenci × PO başına tek satır) ve okuma yükünü hesaplamadan ayırmak için bilinçli olarak denormalize edilmiştir. Kaynak tablolar OLTP tarafıdır; snapshot, öğrenci ve PO boyutlarına bağlı bir olgu tablosu gibi davranır.

### Bu sprintte yapılan düzeltmeler

| Düzeltme | Neden |
|---|---|
| `Enrollment` (öğrenci ↔ ders) tablosu eklendi | Hangi öğrencinin hangi derste olduğu tutulmuyordu |
| `LO_PO_Mapping.weight` tam sayı oldu, 1–5 `CHECK` kısıtı eklendi | Hocanın tanımı 1–5; ondalık alan 0.01'i kabul ediyordu |
| `StudentPOSnapshot.score` NULL olabilir | Verisi olmayan PO'ya 0.00 yazmak öğrenciyi haksız yere düşük gösteriyordu |
| Roller dörde çıktı: `ADMIN`, `DEPT_HEAD`, `INSTRUCTOR`, `STUDENT` | Bölüm Başkanı ile sistem yöneticisi aynı roldeydi |
| Unique kısıtlar adlandırıldı; not 0–100 ve ağırlık > 0 için `CHECK` eklendi | Kural uygulama katmanını atlayan yazmalarda da geçerli olsun |
| Her eşleme tablosuna ters yönlü bileşik index eklendi | Unique index yalnızca bir yönü karşılıyordu |

### Kısıtlar ve index'ler

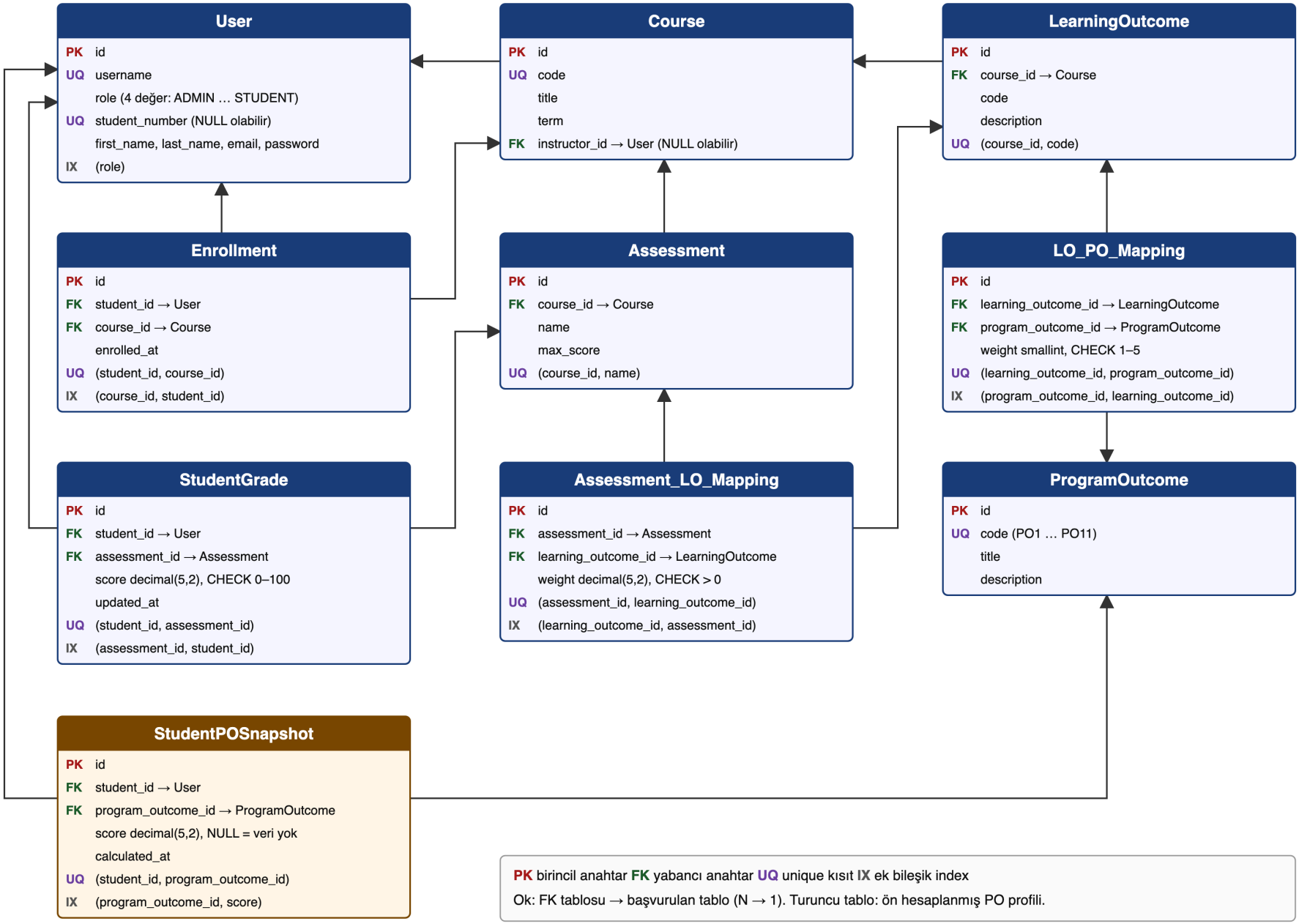
| Tablo | Unique | Check | Ek index |
|---|---|---|---|
| `User` | `username`, `student_number` | — | `(role)` |
| `ProgramOutcome` | `code` | — | — |
| `Course` | `code` | — | — |
| `Enrollment` | `(student, course)` | — | `(course, student)` |
| `LearningOutcome` | `(course, code)` | — | — |
| `LO_PO_Mapping` | `(learning_outcome, program_outcome)` | `weight` 1–5 | `(program_outcome, learning_outcome)` |
| `Assessment` | `(course, name)` | — | — |
| `Assessment_LO_Mapping` | `(assessment, learning_outcome)` | `weight > 0` | `(learning_outcome, assessment)` |
| `StudentGrade` | `(student, assessment)` | `score` 0–100 | `(assessment, student)` |
| `StudentPOSnapshot` | `(student, program_outcome)` | — | `(program_outcome, score)` |

### Tablo boyutları (demo verisi, PostgreSQL 18)

`seed_demo` sonrası `pg_relation_size`, `pg_indexes_size` ve `pg_total_relation_size` ile ölçülmüştür.

| Tablo | Satır | Sütun | Veri (B) | Index (B) | Toplam (B) |
|---|---|---|---|---|---|
| outcomes_user | 11 | 13 | 8 192 | 98 304 | 114 688 |
| outcomes_programoutcome | 11 | 4 | 8 192 | 49 152 | 65 536 |
| outcomes_course | 3 | 5 | 8 192 | 65 536 | 73 728 |
| outcomes_enrollment | 16 | 4 | 8 192 | 81 920 | 90 112 |
| outcomes_learningoutcome | 7 | 4 | 8 192 | 49 152 | 65 536 |
| outcomes_lo_po_mapping | 11 | 4 | 8 192 | 81 920 | 90 112 |
| outcomes_assessment | 7 | 4 | 8 192 | 49 152 | 57 344 |
| outcomes_assessment_lo_mapping | 13 | 4 | 8 192 | 81 920 | 90 112 |
| outcomes_studentgrade | 37 | 5 | 8 192 | 81 920 | 90 112 |
| outcomes_studentposnapshot | 66 | 5 | 32 768 | 81 920 | 139 264 |

Bu hacimde her tablo tek bir 8 KB sayfaya sığar; boyutu index'ler belirler. Tek istisna snapshot tablosudur: 66 satır için dört sayfa kullanır, çünkü seed sırasında her not kaydı aynı satırları yeniden yazmış ve PostgreSQL eski satır sürümlerini (MVCC) henüz temizlememiştir. Gerçekçi hacimli ölçüm (500 öğrenci, 40 ders) final raporunda yapılacaktır.




**Şekil 4 — ER diyagramı**

## 4 Scalability & Fault Tolerance

Bu bölüm bir plandır. Yapılanlar ve yapılacaklar ayrı gösterilmiştir.

### Yük profili

Sistem okuma ağırlıklıdır: not dönemde birkaç kez yazılır, PO profili ise her panel açılışında okunur. Tasarımın ana fikri, pahalı hesabı yazma anına taşıyıp okumayı tek tablodan yapmaktır.

### Yapılanlar

| Teknik | Nerede | Ne kazandırır |
|---|---|---|
| Ön hesaplama | `StudentPOSnapshot` | PO profili üç tablolu join ve toplama yerine tek index'li okumadır |
| Artımlı yeniden hesaplama | `outcomes/signals.py` | Bir değişiklik yalnızca etkilenen öğrencileri yeniden hesaplatır |
| Bileşik B-tree index'ler | Tüm eşleme, not ve snapshot tabloları | Her iki sorgu yönü de (öğrenci → not, değerlendirme → not) index'ten karşılanır |
| Yeniden üretilebilir snapshot | `POCalculatorService.recalculate_students` | Snapshot bozulur ya da silinirse kaynak tablolardan yeniden kurulur |
| Ortamdan bağımsız yapılandırma | `config/settings.py` | Aynı kod SQLite ile dizüstünde, PostgreSQL ile sunucuda çalışır |

### Planlananlar

| Teknik | Ne zaman gerekir | Not |
|---|---|---|
| Redis cache (ders × PO ortalamaları) | Bölüm başkanı paneli geldiğinde | Ayar hazır; cache'lenecek sorgu henüz yok |
| PostgreSQL materialized view (ders × PO) | Isı haritası raporu geldiğinde | Dönem içinde periyodik `REFRESH` |
| Arka plan kuyruğu | Bir ağırlık değişikliği yüzlerce öğrenciyi etkilediğinde | Bugün yeniden hesaplama istek içinde, eşzamanlı çalışır |
| Günlük `pg_dump` yedeği | Canlıya çıkışta | Snapshot yedeklenmese de yeniden üretilebilir |
| Bağlantı havuzu, okuma replikası | Birden çok bölüm kullanırsa | Şimdilik `CONN_MAX_AGE = 60` yeterli |

### Beklenen hacim

500 öğrenci, 40 ders, ders başına 4 değerlendirme ve öğrenci başına dönemde 6 ders varsayımıyla tahmin:

| Tablo | Tahmini satır |
|---|---|
| `StudentGrade` | 500 × 6 × 4 = 12 000 |
| `Enrollment` | 500 × 6 = 3 000 |
| `StudentPOSnapshot` | 500 × 11 = 5 500 |
| Eşleme tabloları | birkaç yüz |

Bu hacim tek bir PostgreSQL sunucusu için küçüktür. Darboğaz veri boyutu değil, toplu yeniden hesaplamadır: çok öğrencili bir dersin LO→PO ağırlığı değişince o dersin tüm öğrencileri aynı transaction içinde yeniden hesaplanır.

### Hata durumları

| Hata | Davranış |
|---|---|
| Hesaplama sırasında istisna | Transaction geri alınır; not ve snapshot eski hâlinde kalır |
| Redis erişilemez | Cache'lenen sorgu olmadığı için bugün etkisi yok; ileride veritabanına düşülecek |
| SMTP erişilemez | E-posta gönderen özellik henüz yok |
| Snapshot tutarsızlığı şüphesi | `recalculate_students` ile tüm öğrenciler yeniden hesaplanır |

## 5 Consistency

Tutarlılık kuralı tektir: snapshot her zaman kaynak tablolardan hesaplanabilecek değere eşit olmalıdır. Bunu üç mekanizma sağlar.

### 1. Değişiklik ve yeniden hesaplama aynı transaction'dadır

Tetikleyiciler kaydın yazıldığı transaction içinde çalışır. `ATOMIC_REQUESTS` açık olduğu için bir HTTP isteğindeki not kaydı ile PO güncellemesi ya birlikte yazılır ya hiç yazılmaz. İstek dışındaki kullanım için servis aynı garantiyi kendisi verir:

```
@classmethod
@transaction.atomic
def save_grade(cls, student, assessment, score):
    grade = (StudentGrade.objects.select_for_update()
             .filter(student=student, assessment=assessment).first()
             ) or StudentGrade(student=student, assessment=assessment)
    grade.score = Decimal(str(score))
    grade.full_clean()     # 0–100 dışı not burada reddedilir
    grade.save()           # post_save sinyali PO'ları yeniden hesaplar
    return grade
```

### 2. Aynı öğrenci için hesaplamalar sıraya girer

PostgreSQL'in varsayılan izolasyon düzeyi READ COMMITTED'dır. Bu düzeyde aynı öğrencinin iki notu aynı anda kaydedilirse, iki hesaplama da diğerinin notunu görmeden snapshot yazabilir. Bunu önlemek için hesaplama önce öğrenci satırını kilitler:

```
@classmethod
@transaction.atomic
def calculate_student_profile(cls, student):
    locked = User.objects.select_for_update().filter(pk=student.pk).first()
    ...
```

İkinci hesaplama birincinin commit'ini bekler ve güncel notları okur. Toplu hesaplamada öğrenciler `pk` sırasıyla kilitlenir; böylece iki toplu işlem birbirini kilitleyemez (deadlock). SQLite'ta `select_for_update` etkisizdir; orada yazmalar zaten tek tek yapılır.

### 3. Veritabanı kısıtları

Uygulama katmanını atlayan yazmalar (toplu yükleme, elle SQL) da kurallara uyar: unique kısıtlar aynı öğrenciye aynı değerlendirmeden iki not yazılmasını, `CHECK` kısıtları 0–100 dışı notu ve 1–5 dışı ağırlığı engeller.

### Hangi değişiklik kimi yeniden hesaplatır

| Değişen kayıt | Yeniden hesaplanan öğrenciler |
|---|---|
| `StudentGrade` (ekleme, güncelleme, silme) | Notun sahibi |
| `Assessment_LO_Mapping` | O değerlendirmeden notu olan öğrenciler |
| `LO_PO_Mapping` | O LO'yu besleyen değerlendirmelerden notu olan öğrenciler |

### Testler

`python manage.py test` 15 testi SQLite ve PostgreSQL 18 üzerinde geçirir: hocanın örneği (LO1 86.0, LO2 82.9, PO1 85.0, PO2 82.9), verisi olmayan PO'nun NULL olması, not/eşleme/ağırlık değişince ve silinince yeniden hesaplama, geçersiz notun reddedilip snapshot'ın değişmemesi.

### Bilinen eksikler

- `Assessment.max_score` hesaplamaya katılmıyor; notlar 0–100 kabul ediliyor.
- `QuerySet.update()` ve `bulk_create()` sinyal üretmez; toplu not yükleme geldiğinde servis açıkça çağrılacak.
- Eşzamanlılık testi (iki bağlantıyla aynı anda yazma) henüz yok; kilit davranışı kod incelemesiyle doğrulandı.

## 6 Teamwork

| Üye | Kod görevi | Diyagram | Rapor bölümü |
|---|---|---|---|
| EmreSndkc | Django iskeleti + ayarlar | Sistem mimarisi | 2 System Design (mimari) + 4 Scalability (plan) |
| brkznqr | Model düzeltmeleri + migration'lar | ER diyagramı | 3 Data Model |
| eminozkanca | Hesaplama servisi + otomatik yeniden hesaplama + testler | Class diyagramı | 2 System Design (class) + 5 Consistency |
| Berkegn | Seed verisi + Django admin paneli | Kullanıcı/rol akış şeması | 1 Introduction + roller tablosu |

Çalışma düzeni: `main` ← `develop` ← `feature/<kısa-ad>`; conventional commit mesajları; her görev sahibinin onayından sonra commit'lenir. Kod, diyagram ve rapor taslakları yapay zekâ desteğiyle (Claude) hazırlanmış, her üye kendi bölümünü gözden geçirip onaylamıştır.